



Hospital

Faites adhérer les parties prenantes avec un POC

Architecte Logiciel P11

Emmy Lemonnier

openclassrooms

Mise à jour des documents d'architecture suite au Proof of Concept

Table des matières

1. Objet du document	4
2. Mises à jour proposées pour la Déclaration des travaux d'architecture	4
Mise en œuvre de la PoC	4
Front-end Vue.js.....	4
Respect des principes	5
2.2 Livrables de travail.....	5
2.3 Critères et procédures d'acceptation	5
Critères retenus pour juger la réussite de la PoC :	5
Procédures d'acceptation :.....	6
3. Mises à jour proposées pour le Document de définition de l'architecture	6
3.1 Architecture métier, données, applications, technologie	6
Présentation du prototype microservices	6
Respect des principes	7
3.2 Analyse de l'écart et prochaines étapes	8
3.3 Conformité RGPD.....	8
3.4 Point sur la sécurité.....	9
Utilisation de chiffrement :.....	9
L'importance d'utiliser un orchestrator en termes de résilience et d'observabilité pour une architecture micro service.....	10
Possibilité pour réaliser des tests :	10
Rôles :	10
Recommandations générales	11
Prochaines étapes :	14
Détail sur l'observabilité et la résilience :	15
Conclusion et utilité de ce document :	15

1. Objet du document

Le présent document vise à **compléter et enrichir** :

- **La Déclaration des travaux d'architecture**, qui formalise le cadre, les livrables et les critères d'acceptation de la preuve de concept (PoC),
- **Le Document de définition de l'architecture**, qui décrit le contexte métier, l'architecture cible, ainsi que l'analyse de l'écart entre l'existant et le futur système.

La PoC, réalisée en **Java Spring Boot** (trois microservices distincts) avec un **front-end Vue.js**, a pour but de valider la faisabilité d'une architecture microservices pour la gestion des hôpitaux, de l'authentification et d'un service de calcul de distance.

2. Mises à jour proposées pour la Déclaration des travaux d'architecture

2.1 Approche architecturale et Processus d'architecture

Dans la section *Approche architecturale* (ou *Processus d'architecture*), on peut ajouter :

Mise en œuvre de la PoC

Dans le cadre de la preuve de concept, nous avons implémenté :

- **Un microservice “User/Auth”** pour gérer l'authentification et les utilisateurs (Java Spring Boot + JWT).
- **Un microservice “Hospital”** pour gérer les informations d'un hôpital (nom, localisation, spécialités éventuelles, etc.) et permettre d'ajouter un hôpital via une API REST.
- **Un microservice “Distance”** pour calculer la distance entre deux emplacements, appelé directement par le microservice Hospital.

Front-end Vue.js

- Un front minimaliste développé en Vue.js permet de présenter l'interface utilisateur :
 - Affichage de la liste d'hôpitaux,
 - Formulaire d'ajout d'hôpital,

- Consultation du détail d'un hôpital
- Gestion d'un login/logout basique pour tester la partie auth.
- Recherche d'un hôpital par critère
- Consulter le profil d'un utilisateur

Respect des principes

- Nous avons appliqué une **séparation des responsabilités** (principe B2), en cloisonnant l'authentification d'une part, la gestion d'un hôpital d'autre part.
- L'intégration et la livraison se font de manière continue (principe B3), avec un pipeline CI/CD (même rudimentaire) lancé à merge request.
- Les tests sont automatisés (principe B4), incluant des tests unitaires et quelques tests d'intégration.

Ces développements ont été réalisés **en mode prototype**, dans un environnement isolé et **sans données réelles** de patients (afin de respecter les contraintes RGPD et la sécurité).

2.2 Livrables de travail

Dans la section *Livrables de travail*, on peut ajouter :

- **Code source** des trois microservices Java Spring Boot + la configuration du front-end Vue.js, hébergés sur un dépôt Git.
- **Pipelines CI/CD** : exécution de builds et de tests automatisés pour chaque service.
- **Rapport de tests** : un résumé des tests unitaires et d'intégration effectués.
- **Présentation PowerPoint** détaillant l'architecture, le fonctionnement de la PoC et les futures recommandations.

L'objectif étant de démontrer la faisabilité, ces livrables attestent d'une première intégration fonctionnelle.

2.3 Critères et procédures d'acceptation

Dans la section *Critères et procédures d'acceptation*, on peut préciser :

Critères retenus pour juger la réussite de la PoC :

1. Vérifier que la **communication entre microservices** (Hospital ↔ Distance) fonctionne en REST, avec un temps de réponse stable sur un petit volume de requêtes.

2. **Authentification opérationnelle** : seul un utilisateur authentifié peut ajouter un hôpital.
3. Mise en place d'une **intégration continue** : tout commit déclenche la compilation et les tests unitaires.
4. **Front Vue.js** consommant l'API de manière fluide, illustrant la faisabilité d'une interface utilisateur.
5. **Mise en place de User : token JWT et authentification.**

Procédures d'acceptation :

- Revue de code et revue d'architecture (hospital, distance, auth) par le comité d'architecture.
- Tests exécutés et validés sur la branche principale.
- Démonstration d'usage concret : connexion, ajout d'un hôpital et appel du microservice Distance depuis le front.

3. Mises à jour proposées pour le Document de définition de l'architecture

3.1 Architecture métier, données, applications, technologie

Dans la partie décrivant l'architecture ou le "Modèle de domaine" (architecture métier, système d'information, etc.), on peut ajouter :

Présentation du prototype microservices

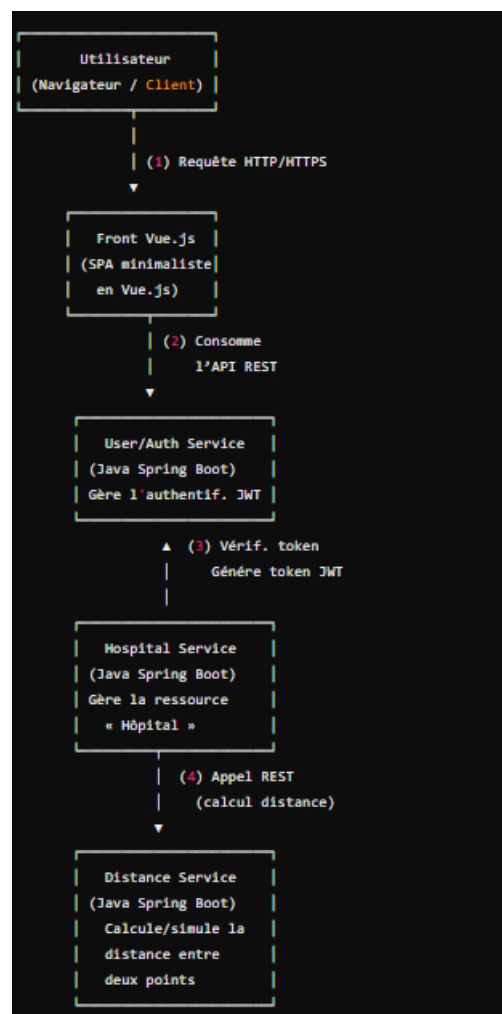
- **Hospital Service** : gère la ressource "Hôpital" (ID, nom, adresse, spécialités, sous spécialité). L'API permet de lister, consulter et ajouter des hôpitaux. À terme, ce service pourrait inclure la gestion de la disponibilité des lits.
- **Distance Service** : appelé par Hospital pour récupérer la distance (fictive ou réelle) entre deux points. Ce service se substitue à un calcul plus poussé qui, dans la version finale, pourrait inclure temps de trajet, contraintes géographiques, et la géo-localisation
- **User/Auth Service** : endosse la fonction "authentification" et "autorisation" pour tous les autres microservices, évitant ainsi tout couplage direct dans Hospital ou Distance.

Technologies mises en œuvre

- **Java Spring Boot** : pour la réalisation de chaque microservice. Les endpoints sont exposés en REST (contrôleurs @RestController).
- **Vue.js** : front-end SPA (Single Page Application) interrogeant les endpoints de l'Auth, Hospital, etc.
- **Mécanismes de sécurité** : JWT (JSON Web Token) pour l'authentification ; un utilisateur doit s'authentifier pour déclencher certaines opérations (ex. ajout d'hôpital).

Respect des principes

- Principes B2 (séparation des préoccupations) et B3 (intégration continue) ont été respectés, validant la mise en place du pipeline.
- Principe B4 (tests automatisés) : la PoC inclut des tests unitaires pour chaque microservice, ainsi que quelques tests d'intégration pour vérifier la communication interservices.



3.2 Analyse de l'écart et prochaines étapes

Dans la section *Analyse de l'écart* ou *Roadmap*, on peut ajouter :

Constats suite à la PoC :

1. **Hospital Service** : les endpoints CRUD de base fonctionnent. Cependant, la gestion de la “disponibilité des lits” n’est pas encore implémentée, et le modèle de données est volontairement simplifié.
2. **Distance Service** : il s’agit d’une implémentation minimaliste (mock ou calcul simpliste). Pour l’architecture cible, il faudra soit intégrer un service de cartographie, soit brancher une API tierce pour des calculs plus fiables.
3. **Authentification** : la PoC démontre la faisabilité d’un microservice Auth. Restent à explorer la gestion fine des rôles, la sécurisation avancée, la tolérance aux pannes, etc.
4. **Front-end Vue.js** : valide la communication asynchrone avec un design modulaire. Son périmètre reste minimal, mais suffisant pour illustrer les appels aux microservices.

3.3 Conformité RGPD

L'architecture microservices peut être conçue de manière à être conforme au Règlement Général sur la Protection des Données (RGPD) en mettant en œuvre les mesures de sécurité appropriées pour protéger les données personnelles des utilisateurs. Cela comprend la mise en place de contrôles d'accès stricts, la sécurisation des communications et des données, ainsi que la mise en place de mécanismes de consentement et de droits des utilisateurs.

Pour rendre conforme la nouvelle architecture aux exigences du RGPD (Règlement Général sur la Protection des Données), voici quelques étapes et bonnes pratiques à suivre :

1. **Analyse des Données** : Identifier les données personnelles collectées, stockées et traitées par le système. Cela peut inclure des informations telles que les noms, les adresses e-mails, les numéros de téléphone, etc.
2. **Minimisation des Données** : Limiter la collecte et le traitement des données personnelles aux seules informations nécessaires à des fins spécifiques. Éviter de collecter des données excessives ou non pertinentes pour le but du traitement.
3. **Consentement et Transparence** : Obtenir le consentement explicite des utilisateurs avant de collecter ou de traiter leurs données personnelles. Fournir des informations claires et compréhensibles sur la manière dont les données seront utilisées, et permettre aux utilisateurs de retirer leur consentement à tout moment.

4. **Sécurité des Données** : Mettre en place des mesures de sécurité robustes pour protéger les données personnelles contre les accès non autorisés, les fuites ou les pertes. Cela peut inclure le chiffrement des données, l'authentification forte, la surveillance des accès, etc.
5. **Droits des Personnes Concernées** : S'assurer que les utilisateurs peuvent exercer leurs droits conformément au RGPD, tels que le droit d'accès, le droit de rectification, le droit à l'effacement, etc. Mettre en place des processus pour répondre rapidement et efficacement à ces demandes.
6. **Gestion des Sous-Traitants** : Veiller à ce que les sous-traitants qui traitent des données personnelles au nom de Web Sonic respectent également les exigences du RGPD. S'assurer de conclure des contrats de traitement des données (DPA) avec ces sous-traitants pour garantir la conformité.
7. **Évaluation d'Impact sur la Protection des Données (EIPD)** : Effectuer une EIPD pour évaluer les risques pour la vie privée associés au traitement des données personnelles et mettre en place des mesures pour atténuer ces risques.
8. **Formation et Sensibilisation** : Sensibiliser et former le personnel sur les principes et les exigences du RGPD, ainsi que sur les meilleures pratiques en matière de protection des données. S'assurer que tous les employés comprennent leur rôle et leurs responsabilités en matière de protection des données.
9. **Conformité avec la CNIL** :
 - **Veille Réglementaire** : Se tenir informé des mises à jour des recommandations, avis et guides pratiques émis par la CNIL.
 - **Dialogue et Collaboration** : Établir un contact régulier avec la CNIL afin de s'assurer que les pratiques mises en œuvre restent en phase avec les évolutions réglementaires.
 - **Déclarations et Autorisations** : Dans certains cas, notamment pour des traitements sensibles, il peut être nécessaire de réaliser une déclaration ou de solliciter une autorisation auprès de la CNIL.
 - **Audits et Contrôles** : Mettre en place des audits internes et, si besoin, des audits en collaboration avec la CNIL pour vérifier le respect des obligations légales et la mise en œuvre effective des mesures de protection.

3.4 Point sur la sécurité

Utilisation de chiffrement :

L'utilisation de HTTPS est cruciale car elle permet de chiffrer les données échangées entre l'utilisateur et le serveur, assurant ainsi que les informations sensibles restent confidentielles et protégées contre les interceptions malveillantes. Contrairement à HTTP, qui transmet les données en clair, HTTPS intègre des protocoles de sécurité qui garantissent l'intégrité et l'authenticité des communications, réduisant considérablement le risque d'attaques de type man-in-the-middle. Adopter HTTPS est donc indispensable pour instaurer un climat de confiance auprès des utilisateurs et

sécuriser les échanges de données dans un environnement numérique de plus en plus vulnérable aux cybermenaces.

L'importance d'utiliser un orchestrator en termes de résilience et d'observabilité pour une architecture micro service

Dans une architecture microservices, un orchestrateur joue un rôle crucial pour assurer la gestion et la coordination efficace de l'ensemble des composants. Par exemple, avec des outils comme Kubernetes ou Docker Swarm, l'orchestrateur permet de :

- **Automatiser le déploiement et la mise à l'échelle** : Il gère le déploiement des conteneurs, assure le scaling horizontal des services en fonction de la charge et peut déployer des mises à jour de manière progressive (rolling updates) sans interruption de service.
- **Assurer la résilience et la haute disponibilité** : En surveillant en continu l'état des conteneurs, l'orchestrateur redémarre automatiquement ceux qui échouent et répartit la charge pour éviter les surcharges sur un seul nœud.
- **Faciliter la gestion du réseau et la découverte des services** : Il configure automatiquement la communication entre les différents microservices, simplifiant ainsi l'interopérabilité et le routage des requêtes.

Possibilité pour réaliser des tests :

ParrotOS, une distribution Linux spécialement conçue pour la sécurité et le test de pénétration, offre un environnement optimal pour simuler des attaques et identifier les vulnérabilités d'une application. Qu'il s'agisse d'une application en phase de preuve de concept (POC) ou d'une application finale, utiliser ParrotOS permet d'exploiter une large gamme d'outils de sécurité (database exploit par exemple) pour détecter et corriger les failles avant qu'elles ne soient exploitées par des acteurs malveillants. Ce processus de test de pénétration est crucial pour renforcer la sécurité globale du système, assurer la protection des données sensibles et instaurer un climat de confiance auprès des utilisateurs, tout en se conformant aux exigences réglementaires en matière de protection des données.

Rôles :

Nous avons pensé à l'attribution des rôles suivant pour l'application finale : Dans notre application, le rôle d'admin représente le super administrateur, chargé de la gestion complète des comptes utilisateurs et de l'administration globale de la plateforme. Ce

compte lui permet de créer, modifier et supprimer des comptes, garantissant ainsi le bon fonctionnement et la sécurité de l'environnement numérique. Par ailleurs, le rôle "user" est attribué aux responsables des hôpitaux, leur offrant un accès spécifique aux fonctionnalités dédiées à la gestion administrative des établissements de santé. Pour répondre aux besoins opérationnels, nous introduisons également un nouveau rôle, "ambulance", qui permettra aux services d'urgence d'utiliser l'application pour rechercher rapidement des hôpitaux selon des critères précis, améliorant ainsi la coordination et l'efficacité dans la prise en charge des situations critiques.

Recommandations générales

Pour mener à bien ce projet nous faisons les recommandations suivantes :

- *Event storming* : il s'agit d'une méthode de modélisation collaborative et visuelle pour concevoir un système informatique.
 - Cette méthode permet à l'équipe de comprendre les processus du métier qui sont impliqués dans un système d'information en modélisant les événements qui se produisent dans le système. On va pouvoir visualiser, comprendre le flux de travail et découvrir les domaines de complexité et identifier les éléments clés du système.
 - La méthode Event Storming implique une collaboration étroite entre toutes les parties prenantes du système : développeurs, chefs de projet, utilisateurs finaux, experts métier. L'ensemble de ces personnes va créer une carte des événements qui se produisent dans le système en utilisant des post-it et des marqueurs pour représenter les différents événements et les relations entre eux.
 - La méthode Event Storming est une méthode itérative et flexible qui est adaptable pour répondre aux besoins spécifiques d'un projet. Elle est donc très populaire au sein de la communauté agile et devops car elle va favoriser la compréhension commune et partager l'ensemble des processus métier et va permettre une collaboration efficace entre les parties prenantes.

Pour finir l'Event Storming permet de comprendre le legacy pour mieux le faire évoluer.

- *DDD Domain driven design* : est une méthode de conception de logiciels qui place la modélisation du domaine métier au cœur du processus de développement.

Cette approche peut aider à créer des systèmes logiciels plus flexibles, évolutifs et pertinents pour les utilisateurs finaux.

- L'objectif de DDD est de créer une représentation de la connaissance du domaine métier au sein du logiciel lui-même. Cela signifie qu'au lieu de simplement coder des fonctionnalités, les développeurs doivent comprendre et modéliser les concepts métiers sous-jacents, tels que les règles de l'entreprise, les processus et les entités clés.
- La méthode DDD encourage également une communication étroite entre les experts métier et les développeurs, afin de s'assurer que le modèle de domaine est précis et pertinent pour les utilisateurs finaux. Le résultat final est un système logiciel qui reflète de manière précise les règles métier, les processus et les exigences, et qui peut être facilement mis à jour et maintenu au fil du temps.
- *Clean Code* : une approche de développement logiciel qui va mettre l'accent sur la lisibilité, maintenabilité du code.
 - Nommage significatif : le nom des variables, fonctions, classes doivent être explicites et évocateur pour faciliter la compréhension
 - Ecriture de fonction courte et ciblée : les fonctions doivent être réduites, cohérentes et se concentrer sur une seule tâche. Leurs noms doivent décrire le fonctionnement.
 - Eviter la duplication du code : un code qui se répète doit être regroupé dans une fonction ou une classe ou un module
 - Utiliser les commentaires intelligemment : le but est d'expliquer pourquoi plutôt que le quoi et le code doit être suffisamment lisible pour ne pas nécessiter de commentaire superflu
 - Utiliser des tests pour valider le code : chaque fonction et module doit être testé pour s'assurer qu'il fonctionne correctement.
 - Respecter les conventions de codage : le code doit être écrit en respectant les conventions de codage standard général et propre à un langage (convention Camel case et convention Microsoft pour C# par exemple) pour assurer la cohérence et faciliter la lecture du code par les autres développeurs.
- *Clean Architecture* : une approche de développement logiciel qui va mettre en avant la séparation des préoccupations et l'indépendance des composants logiciels. On essaie de créer un système modulaire et évolutif en séparant le code en couche fonctionnelles et définies. Chacun a son propre rôle et sa

responsabilité. Cela facilite la modification, maintenance et évolution du système sans compromettre la stabilité et la fiabilité du code

- Indépendance des framework : le code ne doit pas dépendre d'un framework spécifique ou d'une bibliothèque cela facilite la migration vers de nouveaux framework ou technologie.
- Séparer les préoccupations : on sépare en couche les responsabilités du système pour qu'elles soient indépendantes.
- Indépendance des tests : les tests doivent être indépendants des frameworks et bibliothèques pour faciliter leur mise à jour et maintenance
- Indépendance des bases de données : pas de dépendance entre les couches inférieures et supérieures ce qui permet de faciliter le changement de base de données sans impacter le système.
- Limite explicite : les limites entre les couches et composants doivent être explicitées pour faciliter la compréhension et modélisation du système.

Bonnes pratiques générales d'une architecture micro service :

- **Découpage et alignement sur le domaine métier :**
Il faut s'assurer que l'on a bien segmenter l'application en services indépendants, chacun correspondant à une fonctionnalité ou à un domaine métier spécifique. Ce découpage favorise une forte cohésion au sein des services et une faible dépendance entre eux, facilitant ainsi les évolutions et la maintenance.
- **Conception d'API robuste et versionnée :**
Chaque microservice doit exposer des API bien définies et documentées (par exemple, en utilisant OpenAPI/Swagger). La gestion de la version des API permet de garantir la compatibilité entre les services lors des mises à jour ou des évolutions.
- **Gestion des données :**
Il est recommandé que chaque microservice possède sa propre base de données pour éviter les dépendances directes entre services
- **Observabilité et monitoring :**
Mise en place des outils de monitoring, de logs centralisés et de métriques (via Prometheus, Grafana) pour obtenir une visibilité complète sur le comportement de des micro services. Cela facilite le diagnostic et la résolution des incidents.
- **Sécurité et authentification :**
Il faut également s'assurer que la communication entre les microservices est sécurisée (par exemple, en utilisant HTTPS) et que les accès sont bien contrôlés via des mécanismes d'authentification et d'autorisation robustes (comme OAuth2 ou JWT). La gestion centralisée des politiques de sécurité via un API Gateway peut aussi s'avérer très efficace.

- **Déploiement et orchestration :**

L'utilisation de conteneurs (Docker) associée à un orchestrateur (Kubernetes, Docker Swarm) permet d'automatiser le déploiement, la mise à l'échelle et la gestion des services. Cela garantit une haute disponibilité et une meilleure réactivité face aux variations de charge.

- **Automatisation des tests et des déploiements :**

Adoptez une approche CI/CD pour automatiser la validation, le déploiement et le monitoring des microservices. Les tests unitaires, d'intégration et de bout en bout sont essentiels pour garantir la qualité et la stabilité du système dans son ensemble.

Utilisation de OpenAPI :

Utiliser OpenAPI dans ce type d'application offre plusieurs avantages significatifs. Tout d'abord, OpenAPI permet de standardiser et de formaliser la description de l'API, ce qui facilite grandement la compréhension et la consommation de l'interface par les équipes de développement et les partenaires externes. Dans un contexte Java Spring Boot, l'intégration avec des outils comme Springdoc-OpenAPI permet de générer automatiquement une documentation interactive (via Swagger UI), à partir des annotations et du code existant. Cela simplifie non seulement les tests et le débogage des endpoints, mais accélère également le développement en générant des clients ou des stubs automatiquement. Par ailleurs, cette standardisation favorise une meilleure collaboration entre les équipes backend et frontend en clarifiant les attentes et les spécifications des échanges de données, ce qui est particulièrement important dans une architecture microservices où l'interopérabilité est clé. En résumé, OpenAPI contribue à améliorer la transparence, l'efficacité et la maintenabilité de l'API, tout en réduisant les risques d'erreurs de communication entre les différents services.

Prochaines étapes :

- Créer la couche “event-driven” : intégrer un bus d'événements (Kafka, RabbitMQ) pour diffuser les changements d'état (ex. ajout d'un hôpital, changement de disponibilité de lits).
- Gérer des données patient anonymisées : afin de tester l'intégration future de la logique d'attribution de lits, tout en restant conforme à la protection des données.
- Autres micro-services : gestions des lits, prises en compte des symptômes
- Compléter la gestion des hôpitaux : CRUD complet.
- Mettre en place une vraie géo-localisation pour le calcul des distances.

- Observabilité et résilience : outiller l'infrastructure (monitoring, logs centralisés, metrics) et prévoir des mécanismes de fallback en cas de panne (principe B1).
- Sécurité renforcée : finaliser la gestion fine des rôles et autorisations, mettre en place des tests de sécurité (principe B5).

Détail sur l'observabilité et la résilience :

Dans une application Java Spring Boot, il est possible de garantir une observabilité et une résilience optimales en s'appuyant sur des outils intégrés et des bibliothèques dédiées. Par exemple, Spring Boot Actuator fournit des endpoints tels que `/health` et `/metrics` qui offrent une visibilité immédiate sur l'état et la performance de l'application. En combinant Actuator avec Micrometer, vous pouvez exporter ces métriques vers des solutions de monitoring comme Prometheus, puis les visualiser en temps réel avec Grafana. Pour la centralisation des logs, le framework de logging par défaut de Spring Boot (Logback) peut être configuré afin d'envoyer les logs vers une plateforme centralisée comme la stack ELK (Elasticsearch, Logstash, Kibana) ou Graylog, facilitant ainsi l'analyse des incidents et la traçabilité des événements à travers vos microservices.

Du côté de la résilience, l'intégration de bibliothèques telles que **Resilience4j** permet d'implémenter des mécanismes de circuit breaker, de retry et de fallback. Concrètement, si un service en aval rencontre une défaillance répétée, le circuit breaker peut interrompre les appels vers ce service et déclencher une méthode de secours (fallback) pour maintenir la réactivité de l'application. Cette approche évite la propagation de l'erreur dans l'ensemble du système et garantit une continuité de service, même en cas de défaillance partielle. En résumé, en combinant les capacités d'observabilité de Spring Boot Actuator et Micrometer avec les stratégies de résilience offertes par Resilience4j, vous créez un environnement robuste capable de détecter rapidement les anomalies et de réagir efficacement aux incidents, assurant ainsi la stabilité et la performance globale de l'application.

Cette PoC confirme la pertinence d'une architecture microservices et valide certaines options techniques. Elle n'inclut pas encore toutes l'architectures complètes, qui feront l'objet d'itérations ultérieures.

Conclusion et utilité de ce document :

Le présent document peut être remis en complément aux PDF existants ou intégré dans un nouveau livrable Word pour consolider toute la documentation. Il contient l'essentiel

de ce qui a été réalisé dans le cadre du PoC et ce qui devrait apparaître dans les parties pertinentes des deux documents d'architecture.